



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING

SEMESTER 2 2025/2026

SECP3133 HIGH PERFORMANCE DATA PROCESSING

SECTION 01

PROJECT 1

LECTURER: MOHD SHAHIZAN BIN OTHMAN

SHOPPING

Student Name	Matric No.
TAN YI YA	A23CS0187
NURUL ADRIANA BINTI KAMAL JEFRI	A23CS0258
TEH RU QIAN	A23CS0191

Table of Content

1.0 Introduction	3
1.1 Background	3
1.2 Objective	3
1.3 Target Website	3
1.4 Data to be Extracted	3
2.0 System Design and Architecture	4
2.1 System Architecture	4
2.2 Tools and Frameworks Used	5
2.3 Roles of Team Members	6
3.0 Data Collection	6
3.1 Crawling Method	7
3.2 Number of Records Collected	8
3.3 Ethical Considerations	8
4.0 Data Processing	8
4.1 Data Structure	8
4.2 Cleaning Methods	9
4.3 Transformation	12
5.0 Optimization Techniques	14
5.1 Method Used	14
5.2 Code Overview	14
6.0 Performance Evaluation	17
6.1 Before vs after optimization	17
6.2 Time Taken in seconds, CPU Usage (%)	18
6.3 Memory Usage (MB), Throughput (records/s)	18
6.4 Charts and graphs	19
7.0 Challenges and Limitations	20
7.1 Challenges	20
7.2 Limitations	21
8.0 Conclusion and Future Work	21
9.0 References	23
10.0 Appendices	23

1.0 Introduction

1.1 Background

With the exponentially growing amount of online data, web scraping has become one of the techniques for gathering large-scale information for research and analysis purposes. However, processing large volumes of data still remains a significant computational challenge in terms of the time and computational resources needed. Traditional data processing frameworks such as standard single-threaded Python libraries, frequently experience major performance bottlenecks when handling large datasets. This project demonstrates web scraping with Playwright and explores and compares different high-performance data processing methods by using a baseline data processing framework (pandas) and comparing it with modern, parallel computational frameworks (dask and polars) to handle high-throughput workloads efficiently.

1.2 Objective

The objective of this project are stated as below:

1. To design and deploy an asynchronous, effective web crawler to extract over 100,000 records of new articles across different categories.
2. To build multiple data processing pipelines using different frameworks to execute data cleaning, standardisation and meaningful structural transformation.
3. To compare and evaluate the performance of each optimisation framework using core metrics: execution time, CPU usage, memory consumption and throughput

1.3 Target Website

The website selected for this project is malaymail.com, a local news website. This website is selected for its large repository of news articles that traced back to 2010 and well documented data for every category such as title, date, time and author, making it ideal for the project.

1.4 Data to be Extracted

The specific fields targeted for extraction from the news records include:

- Article ID: A unique identifier taken directly from the article's web address URL.
- Title: The headline of the news story.

- Category: The specific editorial section of each article, such as Malaysia, Singapore, Money, World, Life, Eat/Drink, Showbiz, Sports, and Tech/Gadgets.
- Day: Days of the week where each news article is published.
- Date: The date (dd-Mon-yy) where each news article is published.
- Time: The exact timestamp of publication in 12 hours MYT format.
- Author: The name or source attribution of the writer.
- Content: The complete news paragraphs of the news article.
- URL Link: The direct link of the original webpage.

2.0 System Design and Architecture

2.1 System Architecture

Diagram 2.1 shows the data flow:

- Data collection: Playwright scrapes Malay Mail across 9 categories
- Data storage: Raw CSVs are merged and stored on Google Drive as raw_data.csv
- Data processing: The processing pipeline applies the 6 cleaning steps. 3 different tools are used to measure the performance.

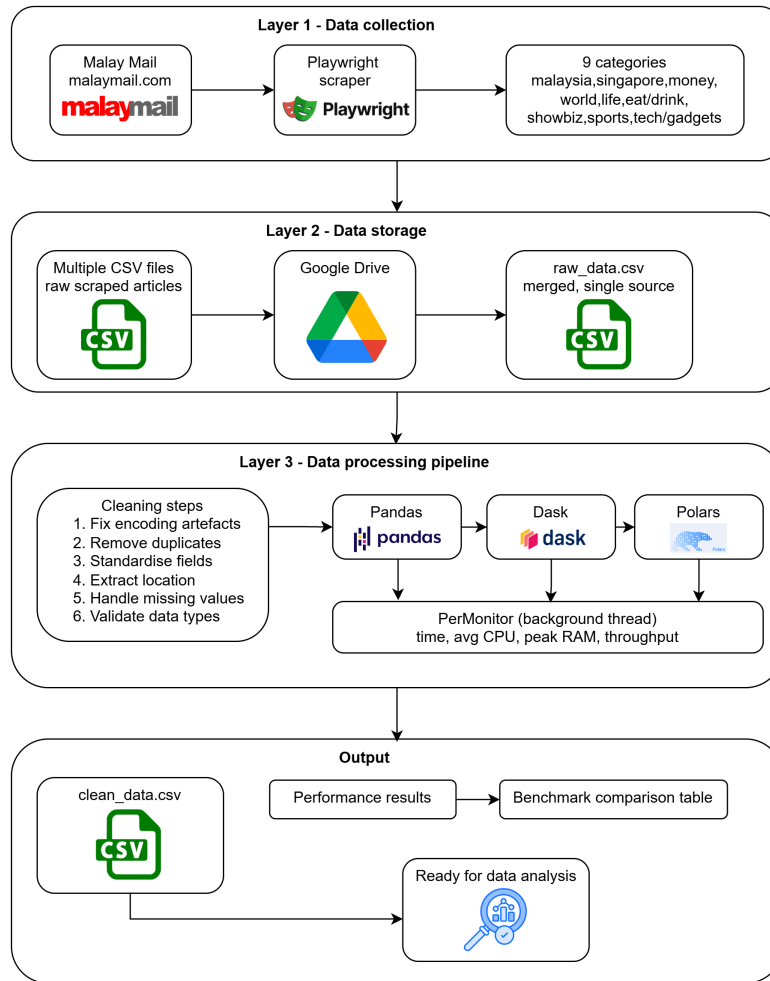


Diagram 2.1 System Architecture

2.2 Tools and Frameworks Used

Diagram 2.2 shows the data flow:

- Data collection: Google Colab runs Playwright scrapes Malay Mail across 9 categories
- Data storage: Raw CSVs are merged and stored on Google Drive as raw_data.csv
- Data processing: Cleaned data are saved as clean_data.csv. 3 different tools are used to measure the performance.
- Presentation: All data and final output are pushed to Github.

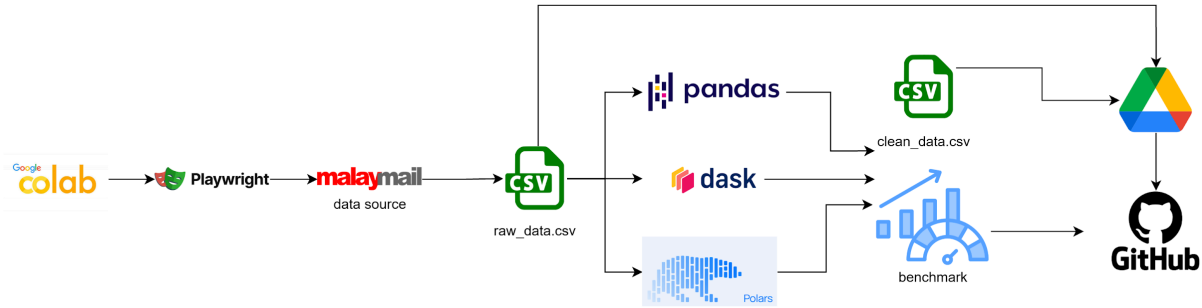


Diagram 2.2 Tools and Frameworks Used

2.3 Roles of Team Members

Table 2.1 Task distribution between group members

Member Name	Task
Adriana	Use Polars for efficient processing, evaluate execution performance
Yi Ya	Implement optimization methods using Dask, optimize workflow performance
Ru Qian	Design system architecture, develop crawling methods, perform baseline data processing using Pandas.

3.0 Data Collection

The data was collected from the news website [Malay Mail](#) by using automated web scraping techniques. The data collection process aimed to collect news articles from multiple categories for analysis and processing. Playwright and Python were used in a Google Colab environment to implement the scraping process. Data collected from each article included the article ID, title, category, publication date and time, author name, article content and URL.

To improve efficiency and reduce the overall execution time, the scraping task was distributed among members. Ru Qian was in charge of the Money, Singapore and Malaysia categories. Yi Ya handled the World, Life and Eat/Drink categories. Adriana managed the Showbiz, Sports and Tech/Gadgets categories.

3.1 Crawling Method

Playwright was chosen because it can effectively handle dynamic web content and provides asynchronous browser automation. The scraper used Chromium in headless mode, which allowed the crawling process to run without opening a graphical browser interface to enhance execution speed.

The crawler started by browsing to the MalayMail website's article listing pages based on the selected news category. Each category page contained multiple article previews, where the scraper extracted the article title and hyperlink from the HTML elements. The URL structure's page number parameter was incremented automatically to allow for continuous crawling across multiple pages.

For each article link identified, the scraper opened the respective article page in a different browser tab and extracted the detailed information. The scraper identified and extracted relevant webpage elements using CSS selectors. Besides, regular expressions (regex) were used to transform data and time values into structured columns and retrieve article IDs from URLs in order to clean and standardize the extracted content.

Using Playwright's asynchronous API and Python's *'asyncio'* library, asynchronous execution was created to increase crawler performance. This made it possible to manage multiple browser activities efficiently without unnecessarily interrupting the execution. Additional waiting mechanisms were implemented to ensure that webpage contents were fully loaded before extraction began.

The crawler design included several recovery and reliable features. First, a *'seen_urls'* set was used to do duplicate URL filtering in order to prevent repetitive article collection. Second, page fingerprinting was used to automatically identify the last page and repeated pagination results. Third, after 50 of articles were collected, scraped data and progress information were constantly saved into CSV and JSON files through the implementation of checkpoint saving.

The current page number, visited URLs and processed page signatures were all saved in the JSON file. This made it possible for the scraper to restart from the most recent successful checkpoint in the case that runtime limitations, internet disconnections or unexpected problems interrupted normal running. Existing CSV files were also reloaded during the restart process to avoid overwriting previously collected data.

To maintain text compatibility, all collected data was exported into CSV format using UTF-8 encoding. The output files were automatically saved to Google Drive to ensure persistent storage beyond the Google Colab runtime environment.

3.2 Number of Records Collected

121,794 of news articles were collected from nine different categories. The categories included Malaysia, Singapore, Money, World, Life, Eat/Drink, Showbiz, Sports and Tech/Gadgets.

3.3 Ethical Considerations

The scraper was conducted to only collect data from the MalayMail website that was available to the public without accessing restricted content. The crawler operated at a controlled rate instead of sending excessive automated traffic and delays were added between requests to reduce the load on the website servers. The collected data was used strictly for academic and research purposes only. Furthermore, the scraping process respected the website's structure and prohibited any actions that could interfere with its functionality. No sensitive data was collected during the scraping process.

4.0 Data Processing

The raw data files collected by members were merged into one single dataset named '*raw_data.csv*'.

4.1 Data Structure

The raw dataset is stored in tabular format using pandas DataFrame.

Column	Description
--------	-------------

id	Unique identifier for each article
title	News title
category	News category
day	Publication day
date	Publication date
time	Publication time
author	Author name
content	Article content
link	Article URL

4.2 Cleaning Methods

A. Encoding Cleaning

Text fields such as *title*, *content* and *author* contained UTF-8 characters. These issues were repaired using *fix_encoding()* function.

Example:

Corrupted Text	Corrected Text
â€™™	'
â€œœ	"
â€"	-

Python

```
def fix_encoding(series):
    s = (series.astype(str))
```

```

        .str.encode('latin1', errors='ignore')
        .str.decode('utf-8', errors='ignore'))

    for bad, good in ENCODING_FIXES:
        s = s.str.replace(bad, good, regex=False)

    return s

```

B. Duplicate Removal

Duplicate records were removed to reduce redundancy.

Python

```

df.drop_duplicates(inplace=True)

df.drop_duplicates(
    subset=['title'],
    keep='first',
    inplace=True
)

```

C. Text Standardisation

Author Cleaning

The prefix “By ” was removed from the author names and invalid entries such as “N/A” and “nan” were converted into missing values.

Example: “By Malay Mail” to “Malay Mail”

Category Cleaning

Values were standardized into lowercase format.

Example: “SPORTS” to “sports”

Python

#Author Cleaning

```
df['author'] = (  
    df['author']  
    .astype(str)  
    .str.replace(r'^By\s+', '', regex=True)  
    .str.strip()  
    .replace({'N/A': None, 'nan': None, '': None})  
)
```

#Category Cleaning

```
df['category'] = (  
    df['category']  
    .astype(str)  
    .str.lower()  
    .str.strip()  
)
```

D. Missing Value Handling

Critical Fields

Rows with missing values in essential fields (*title* and *content*) were removed.

Non-Critical Fields

Missing values in *author*, *location* and *category* were replaced with “Unknown”.

Python

#Critical Fields

```
df.dropna(  
    subset=['title', 'content'],  
    inplace=True  
)
```

```
#Non-Critical Fields
for col in ['author', 'location', 'category']:
    df[col] = df[col].fillna('Unknown')
```

4.3 Transformation

A. Content Parsing and Location Extraction

Regular expression (regex) was used to extract the location and separate it from *content* column

```
Python
#regex
_CONTENT_RE = re.compile(
    r'^([^\, ]+), \s*([A-Za-z]+\s+\d{1,2}) '
    r'(?:\s*[\u2014\u2013-]\s*|\s+)(.*)$'
)

#store into 'location' and 'content' columns
parsed = df['content'].apply(
    lambda x: pd.Series(split_content(x))
)

df['location'] = parsed.iloc[:, 0]
df['content'] = parsed.iloc[:, 1]
```

B. Data Type Transformation

Date Transformation

Date-related columns were converted into datetime format.

String Transformation

Text-based columns were converted into pandas string datatype.

Python

#Date Transformation

```
df[dcol] = pd.to_datetime(  
    df[dcol],  
    errors='coerce'  
)
```

#String Transformation

```
for scol in ['title', 'content',  
            'author', 'category',  
            'location']:  
    df[scol] = df[scol].astype('string')
```

C. Column Reorganisation

The extracted *location* column was repositioned beside the *author* column

Python

```
cols = list(df.columns)  
  
cols.remove('location')  
  
cols.insert(  
    cols.index('author') + 1,  
    'location'  
)  
  
df = df[cols]
```

5.0 Optimization Techniques

5.1 Method Used

5.1.1 Distributed Processing with Dask

Dask is a flexible, open-source parallel computing library written in Python designed to scale analytical workloads from single-core machines to multi-node clusters. Dask overcomes the computational bottleneck of single-core execution by decoupling a program's logical definition from its physical execution. In this project, the data optimization pipeline uses `dask.dataframe` library, which conceptually splits a large, logical dataframe horizontally into multiple smaller Pandas dataframes.

Dask implementation has the following characteristics:

- **Parallel Computing:** Computations run in parallel using multi-processing, which reduces execution time for large datasets.
- **Partitioning:** Datasets larger than memory are processed by small chunks and loaded as needed, which increases efficiency on disk-based datasets.
- **Dynamic Task Scheduling:** Task graph known as Directed Acyclic Graph (DAG) is created to optimize and execute computations dynamically.

5.1.2 High Performance Data Frames: Polars

Polars was originally proposed as a high-performance alternative to the baseline Pandas pipeline, which is limited to a single thread and inefficient with respect to memory usage. Polars is an extremely fast DataFrame library, built in Rust, specifically designed for working with big data.

Three architectural benefits were critical for the selection of Polars to be used in this optimization phase:

- **Built-in Parallelization:** Polars does parallelize operations automatically, unlike Pandas that is limited by Python's Global Interpreter Lock (GIL) and works sequentially on one CPU core. Data transformations like string replacement and regex evaluation are spread uniformly amongst all the available CPU threads, thus drastically decreasing execution time.

- **The Apache Arrow Columnar Memory Format:** This offers cache coherent memory layouts and zero-copy data manipulation. This completely avoids the huge overhead incurred by using Pandas' inefficient object data type in a dataset that is almost all text (articles, titles, authors), therefore significantly reduces the RAM peak usage.
- **Expression Evaluation Engine:** Polars' expression API is very optimized. Polars evaluates expressions concurrently in Rust, instead of executing one at a time while copying intermediate results to memory as Pandas does, which helps cut through on data-cleaning jobs that involve more data.

5.2 Code Overview

5.2.1 Dask

A. Importing Libraries and Load Dataset

Dask library is imported and `dd.read_csv()` loads the CSV as a Dask DataFrame. `blocksize="64MB"` splits the dataset into blocksize partitions for parallel processing.

```
Python
import dask.dataframe as dd

df_raw = dd.read_csv(
    RAW_PATH,
    dtype=str,
    assume_missing=True,
    blocksize="64MB"
)
```

B. Duplicate Removal

Dask performs duplicate removal in parallel across dataframe partitions. This improves scalability for large datasets compared to single-threaded pandas processing.

```
Python
df = df.drop_duplicates()
```

```
if 'title' in df.columns:
    df = df.drop_duplicates(subset=['title'])
```

C. String Cleaning

Dask applies string cleaning operations on each partition concurrently. Parallel execution speeds up text preprocessing for large-scale datasets.

Python

```
for col in ['title', 'content', 'author']:
    if col in df.columns:
        for bad, good in ENCODING_FIXES:
            df[col] = df[col].str.replace(
                bad,
                good,
                regex=False
            )
```

D. Column Standardisation

Dask executes string transformation with lazy execution, by distributing operations across partitions and compute only when triggered.

Python

```
df['author'] = (
    df['author']
    .str.replace(r'(?i)^By\s+', '', regex=True)
    .str.strip()
)
```

E. Apply Function

.apply() executes the custom function on every partition in parallel while meta defines the expected output schema for Dask task planning.

Python

```
extracted = df['content'].apply(  
    split_content_dask,  
    meta=('content', 'object')  
)
```

F. Converting Apply Output into Distributed DataFrame

This section converts extracted dictionary results into structured dataframe columns. Dask distributes the transformation across all partitions.

Python

```
extracted_df = extracted.apply(  
    pd.Series,  
    meta=meta  
)
```

G. Missing Value Handling

Dask processes missing value independently on each partition, which allows scalable cleaning on large datasets.

Python

```
df = df.dropna(subset=critical)  
  
for c in fill_cols:  
    df[c] = df[c].fillna('Unknown')
```

H. Lazy Evaluation with Explicit Execution

This section shows the application of lazy evaluation which `df_clean` operation instead of executing immediately. Computation is triggered by `.persist()` and results are stored in distributed memory for faster reuse.

Python

```
df_clean = clean_dataframe_dask(df_raw)

df_clean = df_clean.persist()
```

I. Count Rows

Application of `.compute()` triggers actual execution of the distributed task graph. Dask aggregates results from all partitions to calculate total rows.

Python

```
n_raw = df_raw.shape[0].compute()
```

5.2.2 Polars

A. Data Ingestion and Typing

Pandas has a loose schema inference where it groups mixed web-scraped data as "objects", while Polars has a strict schema inference. To avoid the pipeline failing on corrupt rows, the raw CSV was fed to the pipeline with the `infer_schema_length=0` flag which forces all columns read from that pipeline to be string types.

```
# Load data (infer_schema_length=0 forces safe string reading)
df_raw = pl.read_csv(RAW_PATH, infer_schema_length=0)
```

B. Encoding Fixes

SLOW STRING MAPPINGS (SLOWSMAP) are used to fix corrupted UTF8 encodings in the baseline. The following trick was used for optimization in Polars, a combination of `with_columns` and `str.replace_all()` functions. All the encoding rules were chained together as an expression list and Polars ran multiple text-replacement rules in parallel on all CPU cores.

```
# A: Fix encoding
```

```

replace_exprs = []
for col in ['title', 'content', 'author']:
    if col in df.columns:
        expr = pl.col(col)
        for bad, good in ENCODING_FIXES:
            expr = expr.str.replace_all(bad, good,
literal=True)
        replace_exprs.append(expr)
if replace_exprs:
    df = df.with_columns(replace_exprs)

```

C. Removing Duplicates

A unique function was used to perform the natively deduplication. A strict subset check was performed using title of article as a source of uniqueness information to ensure data integrity for every batch of the scrapes and to quickly reduce the number of duplicate articles in the data set while keeping the earliest information.

```

# B: Remove duplicates
before = df.height
df = df.unique()
if 'title' in df.columns:
    df = df.unique(subset=['title'], keep='first')
print(f'    Duplicates removed : {before - df.height:,}')

```

D. Standardising Fields

All standardisation in the texts was completely vectorised with Polars expressions. All text standardisation was fully vectorised with Polars expressions. The author column was cleaned up with regex, and other columns such as category and title were converted to lower case and cleaned up to remove extra spaces without having to create mediating dataframes.

```

# C: Standardise fields
if 'author' in df.columns:
    df = df.with_columns(

```

```

        pl.col('author').str.replace(r'(?i)^By\s+', '')
        .str.strip_chars()
        .replace({'N/A': None, 'nan': None,
': None}))
    )
    if 'category' in df.columns:
        df =
df.with_columns(pl.col('category').str.to_lowercase().str.strip_c
hars())
    if 'title' in df.columns:
        df = df.with_columns(pl.col('title').str.strip_chars())

```

E. Location Extraction from Content

The most time consuming part of the baseline was the extraction of the "location" from the "content" column with an apply function – row-wise. This was mapped to return a native pl in order to optimize it. Struct. This meant that the resulting dictionary was unnested in two separate columns and the data was all kept in the fast Rust backend.

```

# D: Extract location from content
if 'content' in df.columns:
    extracted = df.get_column('content').map_elements(
        split_content_polars,
        return_dtype=pl.Struct([pl.Field("location",
pl.String), pl.Field("content", pl.String)])
    )
    df = df.with_columns([
        extracted.struct.field("location").alias("location"),
        extracted.struct.field("content").alias("content")
    ])

cols = df.columns
if 'location' in cols: cols.remove('location')
if 'author' in cols:

```

```
cols.insert(cols.index('author') + 1, 'location')
df = df.select(cols)
```

F. Handling Missing Values (Nulls)

Explicit API of polars was used for dealing with missing data. The `drop_nulls()` function was specifically written to drop records with missing data from critical columns. For non-critical metadata columns, the `.fill_null()` expression was used to efficiently replace the nulls.

```
# E: Handle missing values
critical = [c for c in ['title', 'content'] if c in
df.columns]
before2 = df.height
df = df.drop_nulls(subset=critical)
print(f'      Rows dropped (null critical): {before2 -
df.height:,}')

fill_cols = [c for c in ['author', 'location', 'category'] if
c in df.columns]
if fill_cols:
    df = df.with_columns([pl.col(c).fill_null('Unknown') for
c in fill_cols])
```

G. Validating Data Types

A strict type-casting expression (`cast(pl.String())`) was applied to all text columns that are relevant. This ensured the final data set would be consistent with the baseline data and have absolute data integrity.

```
# F: Validate data types
for scol in ['title', 'content', 'author', 'category',
'location']:
    if scol in df.columns:
        df = df.with_columns(pl.col(scol).cast(pl.String))
```

6.0 Performance Evaluation

6.1 Before vs after optimization

The first version of the pipeline for data processing was created with the standard Pandas library (Baseline Pipeline). Pandas offers powerful data manipulation tools, however, it works with data fully in-memory on a single threaded execution model that can be a big bottleneck with big data sets.

For the optimization of the pipeline we tested two different parallel computing frameworks: Polars and Dask. Polars is built with a lazy-evaluation engine and Apache Arrow memory format, so that tasks will be automatically parallelized. Dask does something different with the original Pandas DataFrame, breaking it into smaller pieces, and using a dynamic task graph to spread the work out across multiple CPU cores. This enabled us to test a brand new engine (Polars) and a distributed version of our baseline engine (Dask).

6.2 Time Taken in seconds, CPU Usage (%)

For 5 runs, the following were observed when switching to parallel programming:

- **Time Taken (s):** The Polars pipeline took 6.21246 seconds to complete the cleaning steps, which is 152.31606 seconds for Dask and 25.6599 seconds for Pandas.
- **CPU Usage (%):** Polars used (on average) 69.36% of CPU while Dask used 70.18% of CPU, versus the Pandas baseline of 59.8%. The average CPU usage in both cases is higher in Polars and Dask indicating multi-threading, that is distributing the work across all cores of the system instead of using one core.

6.3 Memory Usage (MB), Throughput (records/s)

- **Memory Usage (MB):** The peak memory consumption of the baseline Pandas pipeline when run was 2239.7MB, while Polars used 6022.84MB and Dask used 4578.36MB.
- **Optimization:** Polars achieved a significant spike in throughput, reaching 22069.6 records/sec compared to 4813.0 in Pandas. Dask, however, recorded a lower throughput of 811.0 records/sec, reflecting the overhead cost of its task graph scheduling on a moderate-sized dataset. The higher RAM consumption in Polars is attributed to Apache

Arrow pre-loading all data into columnar format simultaneously to enable parallel processing, trading memory for speed.

6.4 Charts and graphs

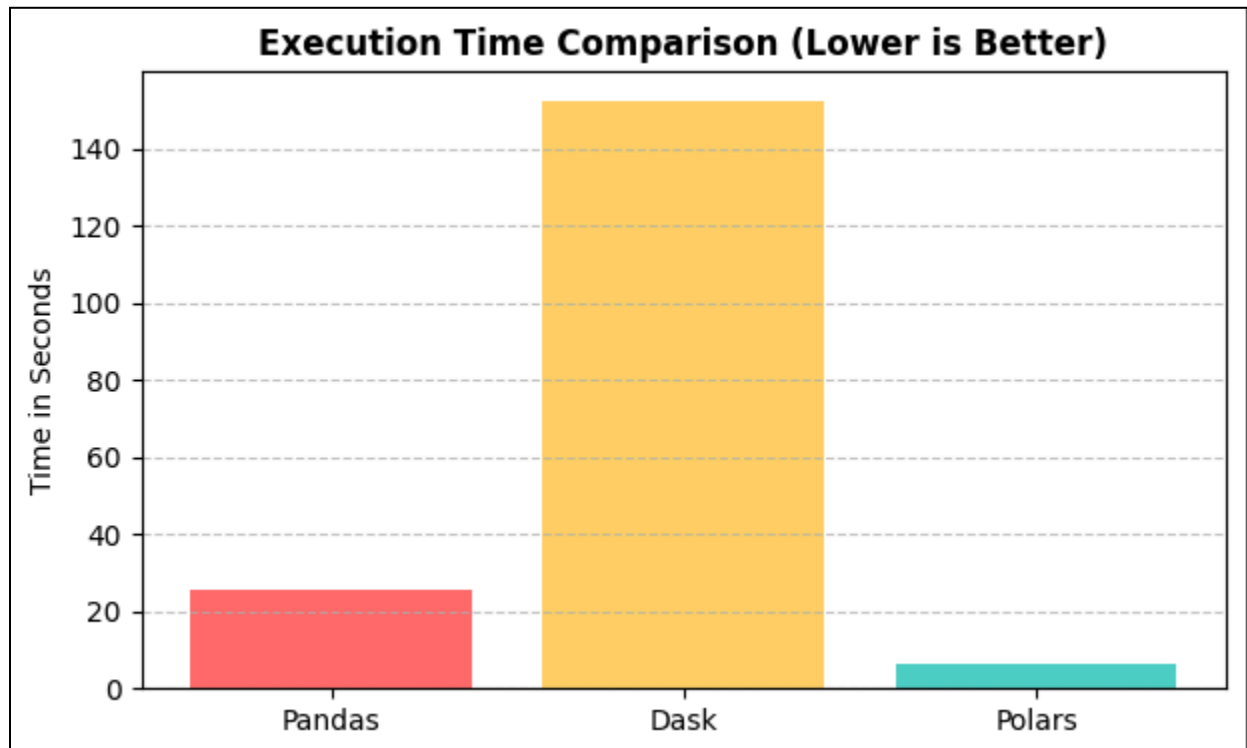


Figure 6.1 Libraries comparison of execution time

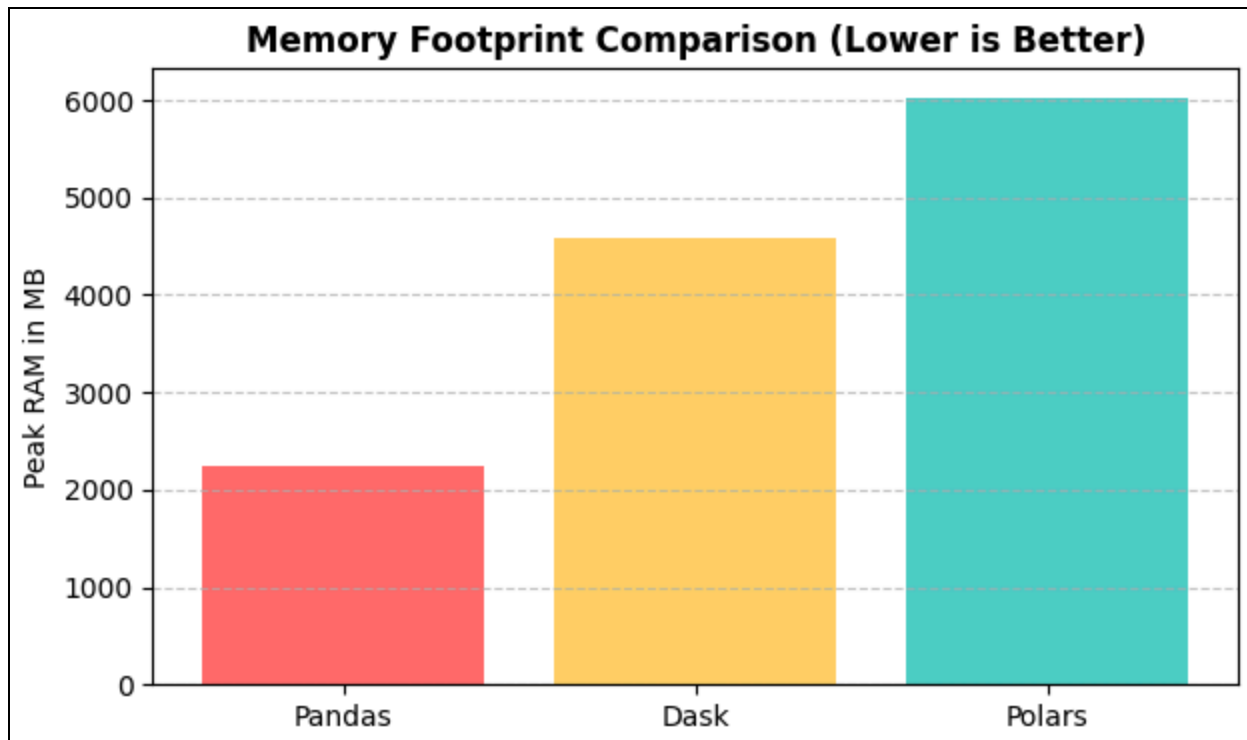


Figure 6.2 Libraries comparison of memories footprint

Table 6.1 An average comparison of measured items using the metric (5-Run Average)

<i>Libraries comparison</i>	Pandas	Dask	Polars
Rows In	123501.0	123501.0	123501.0
Rows Out	121796.0	121796.0	121796.0
Time (s)	25.6599	152.31606	6.21246
Avg CPU (%)	59.8	70.18	69.36
Peak RAM (MB)	2239.7	4578.36	6022.84
Throughput (rps)	4813.0	811.0	22069.6

7.0 Challenges and Limitations

7.1 Challenges

- **The Deep Pagination Wall:** In the first phase of data collection, we went for the category of tech gadgets in the Malay Mail website. But, the server has a hard limit for paging (around page 50). Any attempts to pull more than this led to generic, non-category filler news being added to the server. But a rigorously stringent URL validation filter is needed to prevent corrupted data, and take up the extra space in the higher volume categories such as sports and showbiz (as originally planned) to achieve the requisite number of dataset hits.
- **Colab Memory Exhaustion:** Scraping tens of thousands of articles required opening and closing hidden Chromium browser instances. However it eventually ran out of RAM, causing connection crashes with ``pipe closed by peer``. To get around this, the architecture was modified from a standard scraper to an "Append-Only" engine which appends data in batches, and then safely skips dead pages without going into infinite loops.

7.2 Limitations

- **Network and Bot-Detection Bottlenecks:** The speed of its execution is constrained by the network latency and the target server's anti-bot systems. To simulate the human behavior, and to prevent getting banned from the IP, the sleep timers had to be hardcoded into the script as ``asyncio.sleep()``. In this sense, it is not possible to optimize data collection for speed, as it could lead to server rejections.
- **Limitations with strict Schema Inferences:** Polars offers huge speed optimization, but with the web data we are scraping, it has a minor limitation due to the strict type casting requirement. While Pandas can just classify a mix of data as "objects", Polars needs to be more explicit with its data ingestion rule to get safe string data with the command ``infer_schema_length=0`` in the ``pl.read_csv()`` function.
- **Dask Task Graph Overhead:** Dask is extremely useful when working with out of core computation (data sets larger than the memory of the system). But for the number of data points we scraped and specific to our context, partitioning the data and creating the task graph adds scheduling overhead. The drawback of this is that Dask may not be as fast as

Polars for medium-sized datasets, because the time of handling the parallel workers takes some of the edge off the parallelization itself.

8.0 Conclusion and Future Work

This project successfully designed, implemented and evaluated a high performance data processing pipeline from scraping data, clean to transforming the news articles into analytic-ready data. Across all frameworks, 123501 raw records were successfully processed and produced 121796 cleaned records, which confirms the functionality and reliability of the workflow regardless of the processing techniques.

The evaluation result reveals the difference between each processing technique. Pandas demonstrated limitations in scalability due to its single-threaded execution model and dependence on Python's Global Interpreter Lock (GIL). The framework completed processing in 25.66 seconds with a throughput of 4,813 records per second and consumed 2,239.7 MB of peak RAM. Although memory usage was lower than the parallel frameworks, sequential execution created computational bottlenecks that reduced efficiency for larger-scale workloads.

Dask on the other hand, recorded the slowest execution performance, requiring 152.32 seconds with a throughput of only 811 records per second. It also suffers from substantial computational overhead, reaching 70.18% average CPU utilization and 4,578.36 MB peak RAM usage. These findings indicate that Dask's strengths are more apparent in massive distributed workloads rather than moderate-scale local processing tasks.

Polars achieved the best overall computational performance. Polars completed the pipeline in only 6.21 seconds while achieving the highest throughput of 22,069.6 records per second. Despite recording the highest peak RAM usage at 6,022.84 MB, the framework maximized multi-core CPU utilization efficiently with an average CPU usage of 69.36%, resulting in significantly faster execution compared to both Pandas and Dask. The results demonstrate that Polars is highly optimized for high-speed local analytical workloads where execution performance is prioritized over memory conservation.

In conclusion, the results confirm that data processing techniques are important for modern data engineering pipelines. Polars is the optimal framework for this study, delivering the fastest execution time, highest throughput and most efficient multi-core processing performance, making it the most suitable choice for localized high-performance data processing pipelines.

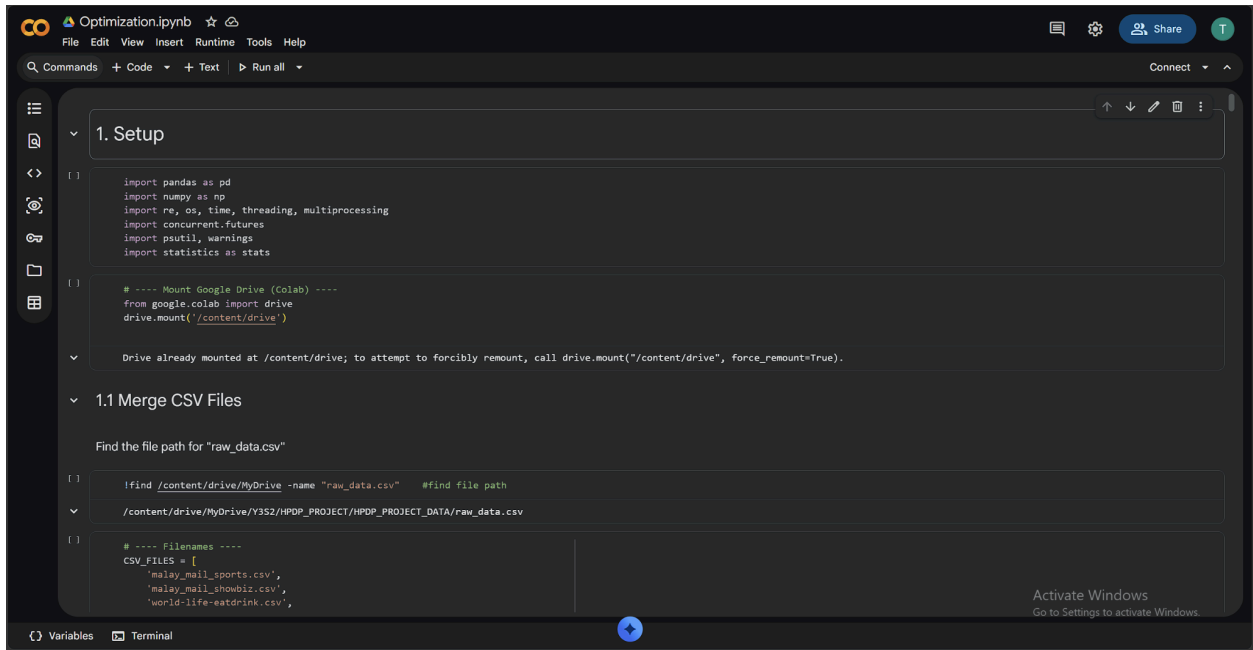
Several expansions could be implemented in future attempts of this project. One of the possible improvements is **moving the current data processing setup to cloud platforms** such as AWS EC2 or Kubernetes clusters. This allows the system to scale horizontally across multiple distributed computers rather than relying entirely on one machine. Another suggestion is changing the pipeline **from batch processing to real-time streaming system** using tools like Apache Kafka. This allows the new article data to be scraped, cleaned and saved as soon as they are published. Lastly, **changing the storing format from csv to Apache Parquet**. Parquet formats are more optimised to improve storage efficiency, reduce file size and speed up future data analysis and business intelligence tasks.

9.0 References

Playwright.dev. (2016). *Installation*. Playwright Python. <https://playwright.dev/python/docs/intro>

10.0 Appendices

Picture 10.0 Screenshot of google colab



The screenshot shows the Google Colab interface with a dark theme. The top bar includes the Colab logo, file name 'Optimization.ipynb', and various menu options like File, Edit, View, Insert, Runtime, Tools, and Help. Below the menu is a search bar and tabs for Commands, Code, Text, and Run all. The left sidebar contains icons for file explorer, code editor, output, and terminal. The main code editor area is divided into sections: '1. Setup' and '1.1 Merge CSV Files'. The '1. Setup' section contains code to import libraries (pandas, numpy, re, os, time, threading, multiprocessing, concurrent.futures, psutil, warnings, statistics) and to mount Google Drive. The '1.1 Merge CSV Files' section contains code to find the file path for 'raw_data.csv' and to list CSV files in a directory. A 'Variables' tab is visible at the bottom left, and a 'Terminal' tab is at the bottom right. A 'Share' button is in the top right corner.

```
import pandas as pd
import numpy as np
import re, os, time, threading, multiprocessing
import concurrent.futures
import psutil, warnings
import statistics as stats

# ---- Mount Google Drive (Colab) ----
from google.colab import drive
drive.mount('/content/drive')

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

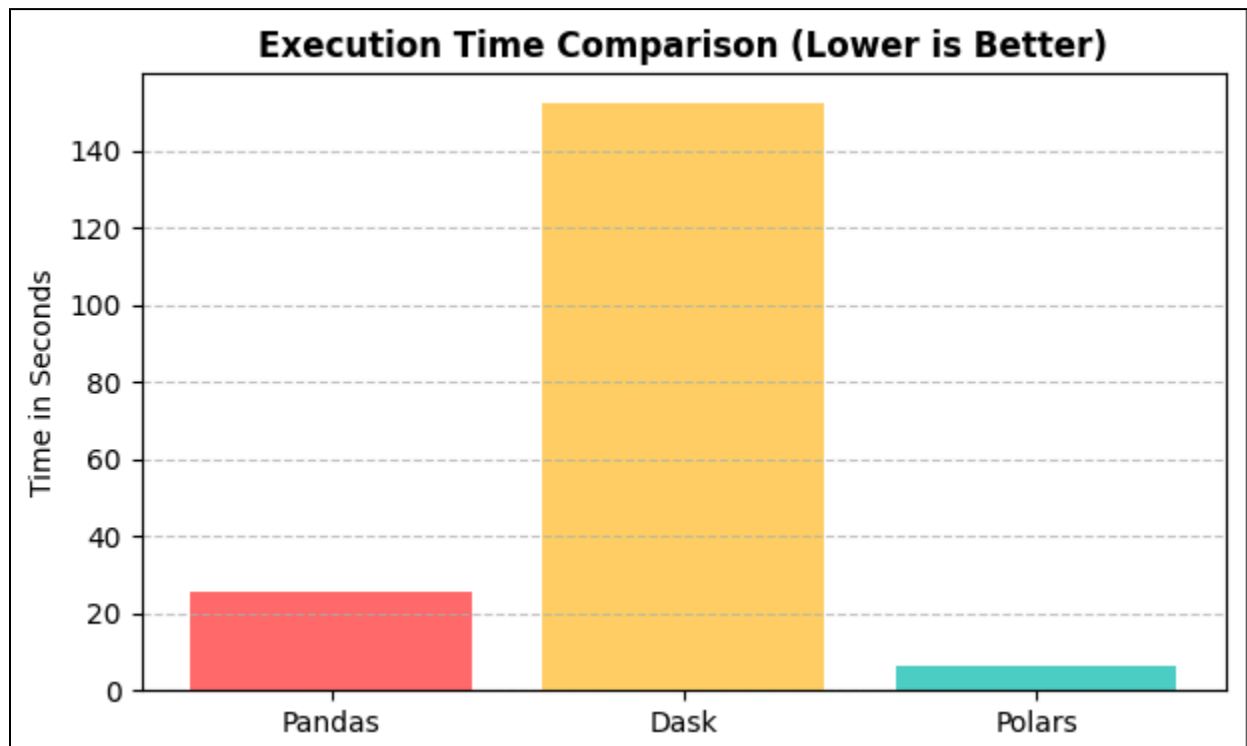
1.1 Merge CSV Files

Find the file path for "raw_data.csv"

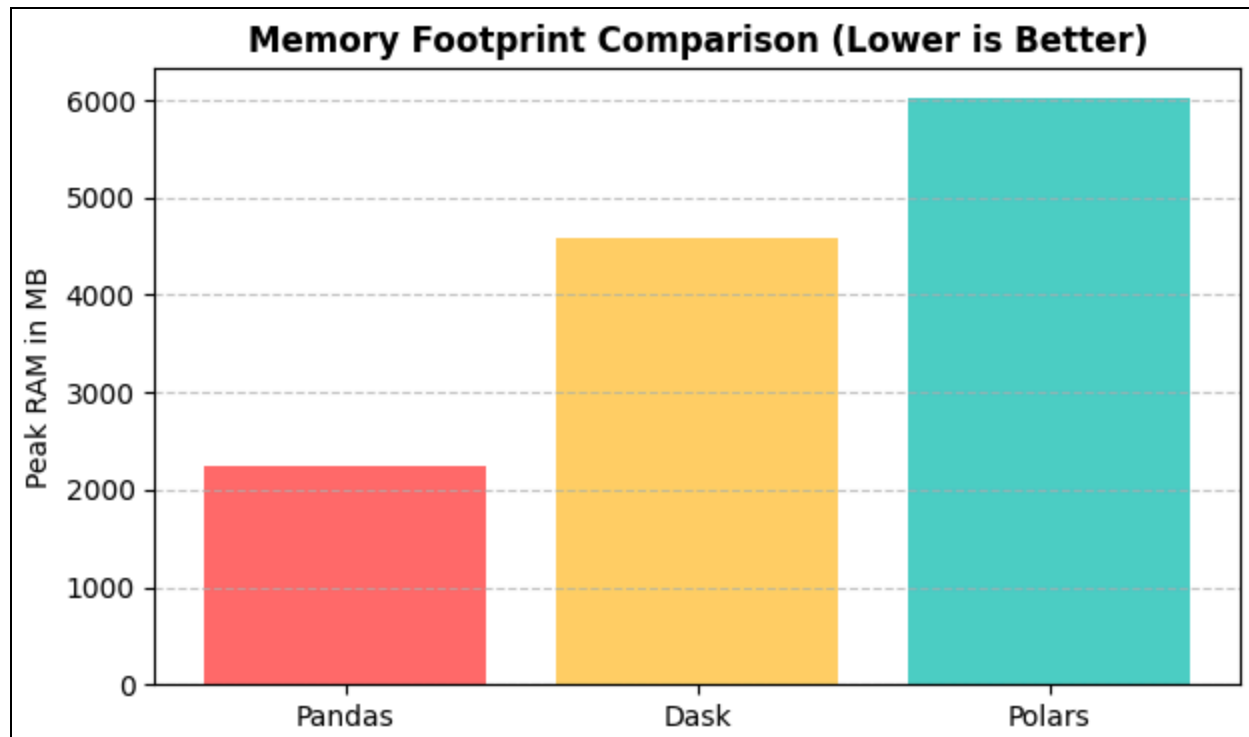
!find /content/drive/MyDrive -name "raw_data.csv" #find file path
/content/drive/MyDrive/Y352/HPDP_PROJECT/HPDP_PROJECT_DATA/raw_data.csv

# ---- Filenames ----
CSV_FILES = [
    'malay_mail_sports.csv',
    'malay_mail_showbiz.csv',
    'world-life-eatdrink.csv',
```

Picture 10.1 Execution time comparison chart between libraries



Picture 10.2 Memory footprint comparison chart between libraries



Link to external sources:

- Google Colab
 - [Web Scraping](#)
 - [Optimisations](#)
- [Github Repository](#)
- [Dataset](#)